

# Python Topic 19 — Type Hints & Pydantic

Full Stack Python + AI Roadmap • Taught in 3 layers: New to Python → Intermediate → Expert • The bridge into FastAPI

## Layer 1 — New to Python

**Type hints** let you label what type a variable, parameter, or return value should be. Python doesn't enforce them at runtime — they're for humans, editors, and tools.

```
name: str = "Akshay"           # this variable should be a string
age: int = 26                   # this should be an integer

def greet(name: str) -> str:    # takes a str, returns a str
    return f"Hello, {name}"
```

The `: str` labels the type, and `-> str` labels what the function returns. It's like TypeScript for Python — annotations that make your code self-documenting and catch mistakes early.

## Layer 2 — Intermediate (real code + how it works)

### TypeScript comparison

```
// TS
function greet(name: string, age: number): string {
    return `${name} is ${age}`;
}

# Python
def greet(name: str, age: int) -> str:
    return f"{name} is {age}"
```

**i Key difference:** TS enforces types at compile time; Python does **not** enforce them at all by default (they're hints).

### Typing collections (built-in in modern Python 3.9+)

```
names: list[str] = ["Akshay", "Riya"]
scores: dict[str, int] = {"math": 90}
coords: tuple[int, int] = (3, 5)

from typing import Optional
```

```
middle_name: Optional[str] = None      # str or None
middle_name: str | None = None          # same thing, cleaner (3.10+)
```

## Common typing constructs

```
from typing import Any, Callable

x: int | None          # int or None (3.10+)
y: int | str            # int OR str (union)
z: Any                 # any type (escape hatch – avoid when possible)
func: Callable[[int], str] # a function taking int, returning str

def process(items: list[dict[str, int]]) -> bool: # nested types
    return len(items) > 0
```

## Type hints don't enforce — proof

```
def add(a: int, b: int) -> int:
    return a + b

add("hello", "world") # ✅ runs! returns "helloworld" – hints NOT enforced
```

⚠️ **Important:** Python won't stop you. Type hints power **tooling** (autocomplete, mypy, IDE errors) but don't change runtime behavior. This is where Pydantic comes in — it *does* enforce.

## 🧠 Layer 3 — Expert (Pydantic — where it gets powerful)

### 1. Pydantic — type hints that actually validate

```
from pydantic import BaseModel

class User(BaseModel):
    name: str
    age: int
    email: str

user = User(name="Akshay", age=26, email="a@b.com") # ✅ valid
user = User(name="Akshay", age="26", email="a@b.com") # ✅ "26" → 26 (coerced!)
user = User(name="Akshay", age="abc", email="a@b.com") # ❌ ValidationError!
```

✅ **Why FastAPI uses it:** unlike plain hints, Pydantic **enforces** types at runtime — validating and coercing input. Incoming JSON gets validated against your model; invalid data is rejected with clear errors before it reaches your

code.

## 2. Validation, defaults, and constraints

```
from pydantic import BaseModel, Field, EmailStr

class User(BaseModel):
    name: str = Field(min_length=1, max_length=50)
    age: int = Field(gt=0, lt=120)           # greater than 0, less than 120
    email: EmailStr                         # validates email format!
    is_active: bool = True                  # default value
    tags: list[str] = []                    # default empty list

User(name="", age=200, email="not-an-email")  # ❌ multiple validation errors
```

`Field` adds constraints (min/max length, ranges), and Pydantic validates emails, URLs, etc. for free — replacing piles of manual `if` checks.

## 3. Nested models

```
class Address(BaseModel):
    city: str
    pincode: str

class User(BaseModel):
    name: str
    address: Address          # nested model!

user = User(name="Akshay", address={"city": "Pune", "pincode": "411001"})
user.address.city            # "Pune" → nested validation happened automatically
```

## 4. Serialization — model ↔ dict ↔ JSON

```
user = User(name="Akshay", age=26)

user.model_dump()           # → dict: {"name": "Akshay", "age": 26}
user.model_dump_json()      # → JSON string
User(**data)                 # dict → model
User.model_validate(data)    # dict → model with validation
```

**i Version note:** `model_dump` is Pydantic v2; older code uses `.dict()`. This request ↔ model ↔ JSON round-trip is what you'll do constantly in APIs.

## 5. The FastAPI connection — why this matters most

```

from fastapi import FastAPI
from pydantic import BaseModel

app = FastAPI()

class CreateUser(BaseModel):
    name: str
    email: EmailStr

@app.post("/users")
async def create_user(user: CreateUser):    # @ automatic validation!
    # FastAPI parses request JSON, validates against CreateUser,
    # and rejects bad input with a 422 error – before this line runs.
    return {"created": user.name}

```

✅ **The killer feature:** declare a Pydantic model as your parameter, and FastAPI automatically parses the body, validates every field, coerces types, returns clean 422 errors, and generates OpenAPI docs. Write the model once; validation, docs, and typing come free. Coming from Express + Zod, Pydantic is like Zod + TypeScript fused into one, built into the framework.

## 6. Custom validators

```

from pydantic import BaseModel, field_validator

class User(BaseModel):
    name: str
    password: str

    @field_validator("password")
    @classmethod
    def check_strength(cls, v: str) -> str:
        if len(v) < 8:
            raise ValueError("Password too short")
        return v

```

Custom validators (a decorator — Topic 16) let you add any rule. Perfect for auth/security work — enforce your "validate on the server" principle right in the model.

## 7. **mypy** — static type checking (for your own code)

```

pip install mypy
mypy myfile.py    # checks type hints WITHOUT running the code

```

`mypy` catches type errors before runtime by analyzing your hints — the closest thing to TypeScript's compiler. Use it in CI to catch bugs early.

## Interview Q&A Cards

### Q: Are Python type hints enforced at runtime?

A: No. They're annotations for tooling (`mypy`, IDEs, autocomplete) and documentation. Code runs even if types are violated — unless you use something like `Pydantic` that validates explicitly.

### Q: What does `Pydantic` add over plain type hints?

A: Runtime validation and coercion. A `BaseModel` checks incoming data against the annotated types, converts where sensible ( `"26"` → `26` ), and raises `ValidationError` on bad input.

### Q: Why is `Pydantic` central to `FastAPI`?

A: `FastAPI` uses `Pydantic` models to auto-parse and validate request bodies, coerce types, return 422 errors for bad input, and generate OpenAPI docs — all from one model definition.

### Q: How do you add a custom rule to a field?

A: With `@field_validator("field")` on a classmethod that receives the value, checks it, and either returns it or raises `ValueError`.

 **Memory hook:** *Type hints = TypeScript for Python, but NOT enforced (just tooling). Pydantic = hints that DO enforce (validate + coerce). FastAPI = Pydantic models auto-validate requests. Pydantic ≈ Zod + TS fused in.*

✅ **One-liner for interviews:** "Type hints annotate expected types but aren't enforced at runtime — they power tooling like `mypy` and IDEs. `Pydantic` uses those same hints to actually validate and coerce data at runtime, which is why `FastAPI` relies on it: request bodies are auto-validated against `BaseModel` classes. Use `Field` for constraints and `@field_validator` for custom rules."